

UNIVERSITÉ CATHOLIQUE DE L'OUEST D'ANGERS

Faculté des Sciences

RAPPORT DE PROJET

Algorithme et Programmation

Planification de la récolte agricole

Réalisé par :

BAH Mohamed Lamine

BARRY Thierno Ousmane

Année universitaire 2025 – 2026

Date de rendu : 31 mai 2026

1. Introduction

Ce rapport présente la conception et la réalisation d'un programme Python permettant de générer automatiquement un emploi du temps de récolte pour un agriculteur (Bertrand MALEAU) et son fils (Louis MALEAU). L'objectif principal est de récolter l'ensemble des 60 champs disponibles en minimisant le nombre de jours nécessaires, tout en respectant les contraintes horaires propres à chaque travailleur.

Le projet a été développé dans le cadre du cours d'Algorithmique et Programmation de la Licence 1 à l'UCO d'Angers, par binôme. Il met en pratique les concepts fondamentaux de la programmation orientée objet en Python : classes, encapsulation, et manipulation de structures de données.

2. Problématique et contraintes

2.1 Présentation du problème

L'exploitation agricole dispose de 60 champs dont les surfaces varient de 1 à 48 hectares. Deux travailleurs doivent récolter l'ensemble de ces champs dans un minimum de jours. Chaque travailleur possède des horaires et une efficacité différents, ce qui implique que la durée de récolte d'un même champ diffère selon le travailleur qui l'effectue.

2.2 Données des travailleurs

ID	Prénom	Nom	Début	Fin	Efficacité (ha/h)
1	Bertrand	MALEAU	7h00	20h00	8 ha/h
2	Louis	MALEAU	9h00	17h00	5 ha/h

2.3 Contraintes à respecter

- Chaque travailleur commence et termine sa journée aux heures définies dans la table WORKERS.
- Une pause déjeuner obligatoire d'une heure doit être prise entre 12h et 13h.
- Un champ ne peut être récolté qu'en une seule et unique fois : il n'est pas possible de commencer un champ le matin et de le terminer l'après-midi.
- Il peut arriver qu'une partie d'une journée reste vide pour un ou plusieurs travailleurs.

3. Réflexion et approche algorithmique

3.1 Découpage de la journée en créneaux

La contrainte d'indivisibilité des champs combinée à la pause déjeuner obligatoire nous a conduit à diviser chaque journée de travail en deux créneaux distincts :

- Le créneau matin : de l'heure de début du travailleur jusqu'à 12h00.
- Le créneau après-midi : de 13h00 jusqu'à l'heure de fin du travailleur.

Ainsi, pour Bertrand (7h-20h), le matin représente 5 heures disponibles et l'après-midi 7 heures. Pour Louis (9h-17h), le matin représente 3 heures et l'après-midi 4 heures.

3.2 Stratégie d'affectation (algorithme glouton)

L'algorithme adopté est un algorithme glouton (greedy) : à chaque jour, pour chaque travailleur, on tente d'affecter un maximum de champs en les traitant du plus grand au plus petit. Ce tri par ordre décroissant de surface garantit que les champs les plus difficiles à placer (les plus grands) sont traités en priorité, ce qui optimise l'utilisation du temps disponible.

Pour chaque champ, on vérifie s'il tient dans le créneau matin restant. Si oui, on l'y affecte. Sinon, on vérifie s'il tient dans le créneau après-midi. Si aucun créneau ne convient, le champ est reporté au jour suivant.

3.3 Calcul des temps de récolte

Le temps de récolte d'un champ par un travailleur est calculé selon la formule :

$$\text{temps_récolte} = \text{hectares} / \text{efficacité}$$

Par exemple, Bertrand met 6h pour récolter un champ de 48 hectares ($48 / 8 = 6h$), tandis que Louis mettrait 9,6h pour le même champ ($48 / 5 = 9,6h$), ce qui dépasse sa journée disponible.

4. Architecture du code

4.1 Vue d'ensemble

Le programme est organisé en trois classes principales, chacune ayant une responsabilité distincte, conformément aux principes de la programmation orientée objet. Une fonction `main()` orchestre l'ensemble du programme.

Classe / Fonction	Rôle	Fichier source
Fields	Gestion des données des champs (lecture CSV, accesseurs)	fields.csv
Workers	Gestion des données des travailleurs (lecture CSV, accesseurs)	workers.csv

Calculs	Algorithme de planification, calculs de temps, génération des graphiques	—
main()	Point d'entrée du programme, affichage des résultats	main.py

4.2 Classe Fields

La classe Fields est responsable du chargement et de l'accès aux données des 60 champs agricoles depuis le fichier fields.csv.

```
class Fields:
    def __init__(self, filename='fields.csv'):
        self.fields_data = {} # {field_id: {id, hectares}}
        self.load_data(filename)

    def load_data(self, filename):
        with open(filename, 'r', encoding='utf-8') as file:
            reader = csv.DictReader(file, delimiter=',')
            for row in reader:
                field_id = int(row['ID'])
                self.fields_data[field_id] = {
                    'id': field_id,
                    'hectares': float(row['HECTARES'])
                }
```

Les méthodes accesseurs permettent de récupérer les informations d'un champ par son identifiant :

- `get_field(field_id)` : retourne le dictionnaire complet d'un champ.
- `get_hectares(field_id)` : retourne uniquement la surface en hectares.
- `get_all_fields()` : retourne tous les champs.
- `get_total_fields()` : retourne le nombre total de champs chargés.

4.3 Classe Workers

La classe Workers gère les informations des travailleurs chargées depuis le fichier workers.csv. Elle stocke pour chaque worker son identifiant, ses prénom et nom, ses horaires de travail et son efficacité.

```
class Workers:
    def __init__(self, filename='workers.csv'):
        self.workers_data = {}
        self.load_data(filename)
```

```
def load_data(self, filename):
    with open(filename, 'r', encoding='utf-8') as file:
        reader = csv.DictReader(file, delimiter=',')
        for row in reader:
            worker_id = int(row['ID'])
            self.workers_data[worker_id] = {
                'id': worker_id,
                'first_name': row['FIRST_NAME'],
                'last_name': row['LAST_NAME'],
                'start_day': int(row['START_DAY']),
                'end_day': int(row['END_DAY']),
                'efficiency': float(row['EFFICIENCY'])
            }
```

Les méthodes accesseurs permettent d'accéder individuellement à chaque attribut d'un worker (get_worker, get_first_name, get_last_name, get_start_day, get_end_day, get_efficiency, get_all_workers).

4.4 Classe Calculs

La classe Calculs est le coeur algorithmique du programme. Elle dépend des instances Fields et Workers et regroupe toutes les fonctions de calcul.

4.4.1 calculer_temps_recolte(field_id, worker_id)

Calcule le temps nécessaire pour qu'un worker récolte un champ donné, en divisant la surface par l'efficacité du worker.

```
def calculer_temps_recolte(self, field_id, worker_id):
    hectares = self.fields.get_hectares(field_id)
    efficiency = self.workers.get_efficiency(worker_id)
    if hectares is None or efficiency is None:
        return None
    return hectares / efficiency # temps en heures
```

4.4.2 calculer_heures_disponibles_par_jour(worker_id)

Calcule le nombre total d'heures de travail effectif par jour en déduisant une heure de pause déjeuner : (heure_fin - heure_debut) - 1.

4.4.3 generer_emploi_du_temps()

Fonction principale de planification. Elle implémente l'algorithme glouton décrit en section 3. Les champs sont triés par surface décroissante, puis affectés jour par jour dans les créneaux matin et après-midi disponibles.

```
def generer_emploi_du_temps(self):
    emploi_du_temps = [] # [worker_id, field_id, jour]
    fields_restants = sorted(self.fields.get_all_fields().keys(),
                             key=lambda f: self.fields.get_hectares(f),
                             reverse=True) # tri décroissant

    jour = 1
    while fields_restants:
        progres = False
        for worker_id in workers_ids:
            heures_matin = 12 - start
            heures_apres_midi = end - 13
            for field_id in fields_restants.copy():
                if temps_matin_utilise + temps_field <= heures_matin:
                    emploi_du_temps.append([worker_id, field_id, jour])
                    ...
                elif temps_apres_midi_utilise + temps_field <= heures_apres_midi:
                    emploi_du_temps.append([worker_id, field_id, jour])
                    ...
            jour += 1
    return emploi_du_temps
```

Le résultat est une liste de listes de la forme [worker_id, field_id, jour], comme le demande le sujet.

4.4.4 calculer_nombre_jours_total(emploi_du_temps)

Détermine le nombre total de jours du planning en lisant le jour maximum présent dans le tableau de résultats.

```
def calculer_nombre_jours_total(self, emploi_du_temps):
    if not emploi_du_temps:
        return 0
    return max(assignment[2] for assignment in emploi_du_temps)
```

5. Résultats obtenus

5.1 Exécution du programme

Le programme a été exécuté avec succès. Les fichiers CSV ont été lus correctement : 60 champs et 2 workers ont été chargés. L'algorithme a généré un emploi du temps complet en 14 jours.

```

C:\Users\456706\PyCharmMiscProject\.venv\Scripts\python.exe "C:\Users\456706\Documents\projet algo2\projet algo2\main2.py"
colonnes détectées: ['ID', 'HECTARES']
colonnes détectées: ['ID', 'FIRST_NAME', 'LAST_NAME', 'START_DAY', 'END_DAY', 'EFFICIENCY']
=====
Voici les résultats attendus
=====

Champs chargés: 60
travailleurs chargés: 2

Génération de l'emploi du temps optimal

=====
Emploi du temps optimal:
=====

Nombre total d'affectations: 60
Nombre de jours total: 14

```

Figure 1 – Résumé de l'exécution : chargement des données et résultat global

5.2 Détail des affectations

Le tableau ci-dessous présente un extrait des affectations générées. Chaque ligne indique le travailleur, le champ et le jour d'affectation, ainsi que la surface et la durée de récolte correspondante.

```

Détails des affectations [Worker_ID, Field_ID, Jour]:
=====
1. [Worker 1, Field 2, Jour 1] - Bertrand MALEAU récolte 48.0ha en 6.00h
2. [Worker 1, Field 6, Jour 1] - Bertrand MALEAU récolte 39.0ha en 4.88h
3. [Worker 1, Field 14, Jour 1] - Bertrand MALEAU récolte 8.0ha en 1.00h
4. [Worker 1, Field 7, Jour 1] - Bertrand MALEAU récolte 1.0ha en 0.12h
5. [Worker 2, Field 4, Jour 1] - Louis MALEAU récolte 20.0ha en 4.00h
6. [Worker 2, Field 41, Jour 1] - Louis MALEAU récolte 15.0ha en 3.00h
7. [Worker 1, Field 20, Jour 2] - Bertrand MALEAU récolte 48.0ha en 6.00h
8. [Worker 1, Field 25, Jour 2] - Bertrand MALEAU récolte 37.0ha en 4.62h
9. [Worker 1, Field 40, Jour 2] - Bertrand MALEAU récolte 8.0ha en 1.00h
10. [Worker 1, Field 38, Jour 2] - Bertrand MALEAU récolte 3.0ha en 0.38h
11. [Worker 2, Field 24, Jour 2] - Louis MALEAU récolte 18.0ha en 3.60h
12. [Worker 2, Field 48, Jour 2] - Louis MALEAU récolte 15.0ha en 3.00h
13. [Worker 2, Field 21, Jour 2] - Louis MALEAU récolte 2.0ha en 0.40h
14. [Worker 1, Field 50, Jour 3] - Bertrand MALEAU récolte 48.0ha en 6.00h
15. [Worker 1, Field 30, Jour 3] - Bertrand MALEAU récolte 35.0ha en 4.38h
16. [Worker 1, Field 9, Jour 3] - Bertrand MALEAU récolte 7.0ha en 0.88h
17. [Worker 1, Field 16, Jour 3] - Bertrand MALEAU récolte 5.0ha en 0.62h
18. [Worker 1, Field 8, Jour 3] - Bertrand MALEAU récolte 1.0ha en 0.12h
19. [Worker 2, Field 34, Jour 3] - Louis MALEAU récolte 18.0ha en 3.60h
20. [Worker 2, Field 52, Jour 3] - Louis MALEAU récolte 14.0ha en 2.80h
21. [Worker 2, Field 49, Jour 3] - Louis MALEAU récolte 1.0ha en 0.20h
22. [Worker 1, Field 58, Jour 4] - Bertrand MALEAU récolte 46.0ha en 5.75h
23. [Worker 1, Field 29, Jour 4] - Bertrand MALEAU récolte 34.0ha en 4.25h
24. [Worker 1, Field 10, Jour 4] - Bertrand MALEAU récolte 10.0ha en 1.25h
25. [Worker 1, Field 5, Jour 4] - Bertrand MALEAU récolte 6.0ha en 0.75h
26. [Worker 2, Field 44, Jour 4] - Louis MALEAU récolte 18.0ha en 3.60h
27. [Worker 2, Field 11, Jour 4] - Louis MALEAU récolte 12.0ha en 2.40h
28. [Worker 1, Field 22, Jour 5] - Bertrand MALEAU récolte 45.0ha en 5.62h
29. [Worker 1, Field 42, Jour 5] - Bertrand MALEAU récolte 34.0ha en 4.25h
30. [Worker 1, Field 33, Jour 5] - Bertrand MALEAU récolte 9.0ha en 1.12h
31. [Worker 1, Field 45, Jour 5] - Bertrand MALEAU récolte 5.0ha en 0.62h
32. [Worker 2, Field 13, Jour 5] - Louis MALEAU récolte 16.0ha en 3.20h
33. [Worker 2, Field 46, Jour 5] - Louis MALEAU récolte 12.0ha en 2.40h
34. [Worker 2, Field 1, Jour 5] - Louis MALEAU récolte 4.0ha en 0.80h
35. [Worker 1, Field 23, Jour 6] - Bertrand MALEAU récolte 45.0ha en 5.62h

```

Figure 2 – Extrait du tableau des affectations (jours 1 à 4)

```

=====
Résultat sous forme de tableau (liste de listes):
[[1, 2, 1], [1, 6, 1], [1, 14, 1], [1, 7, 1], [2, 4, 1], [2, 41, 1], [1, 20, 2], [1, 25, 2], [1, 40, 2], [1, 30, 2], [2, 24, 2], [2,
=====

Génération des graphiques...

Graphique sauvegardé: emploi_du_temps.png
Graphique détaillé sauvegardé: planning_detaillé.png
=====

Projet terminé avec succès!
=====

Process finished with exit code 0

```

Figure 3 – Fin du tableau et résultat final sous forme de liste de listes

5.3 Analyse des résultats

L'algorithme a réparti les 60 champs sur 14 jours de travail. On observe que Bertrand, grâce à ses horaires plus étendus (7h-20h) et à sa meilleure efficacité (8 ha/h), prend en charge la majorité des champs. Louis, dont la plage horaire est plus courte (9h-17h) et l'efficacité moindre (5 ha/h), complète les créneaux avec des champs de taille adaptée.

Les champs les plus grands (48 ha) sont systématiquement traités en premier et tiennent dans un seul créneau pour Bertrand ($48 / 8 = 6h$, inférieur aux 5h du matin... ils sont donc placés en après-midi ou matin selon les jours). Les petits champs d'1 hectare sont utilisés pour remplir les créneaux résiduels.

6. Lexique des variables, fonctions et classes

6.1 Classes

Classe	Description
Fields	Représente l'ensemble des champs agricoles. Charge les données depuis fields.csv et fournit des accesseurs par ID.
Workers	Représente l'ensemble des travailleurs. Charge les données depuis workers.csv et fournit des accesseurs par ID.
Calculs	Regroupe toutes les fonctions de calcul : temps de récolte, génération de l'emploi du temps, comptage des jours, génération des graphiques.

6.2 Variables principales

Variable	Type	Description
fields_data	dict	Dictionnaire {field_id: {id, hectares}} stockant tous les champs.
workers_data	dict	Dictionnaire {worker_id: {id, first_name, last_name, start_day, end_day, efficiency}} stockant tous les workers.
emploi_du_temps	list	Liste de listes [worker_id, field_id, jour] représentant le planning complet.
fields_restants	list	Liste des IDs de champs non encore récoltés, triée par surface décroissante.
jour	int	Compteur du jour courant dans la boucle de planification.
temps_matin_utilise	float	Cumul des heures déjà affectées sur le créneau matin d'un worker pour le jour courant.
temps_apres_midi_utilise	float	Cumul des heures déjà affectées sur le créneau après-midi d'un worker pour le jour courant.
heures_matin	int	Nombre d'heures disponibles dans le créneau matin d'un worker (12 - start_day).
heures_apres_midi	int	Nombre d'heures disponibles dans le créneau après-midi d'un worker (end_day - 13).
progres	bool	Indicateur booléen : True si au moins un champ a été affecté dans le jour courant, False sinon (sécurité anti-boucle infinie).
hectares	float	Surface d'un champ en hectares.
efficiency	float	Nombre d'hectares récoltés par heure par un worker.
temps_field	float	Durée nécessaire pour récolter un champ donné par un worker donné (en heures).

6.3 Fonctions

Fonction	Description
load_data(filename)	Lit un fichier CSV et remplit le dictionnaire de données correspondant.
get_field(field_id)	Retourne le dictionnaire complet d'un champ via son ID.
get_hectares(field_id)	Retourne la surface en hectares d'un champ.
get_all_fields()	Retourne tous les champs sous forme de dictionnaire.
get_total_fields()	Retourne le nombre total de champs chargés.
get_worker(worker_id)	Retourne le dictionnaire complet d'un worker via son ID.
get_efficiency(worker_id)	Retourne l'efficacité (ha/h) d'un worker.
calculer_temps_recolte(f, w)	Calcule le temps de récolte d'un champ f par un worker w (hectares / efficiency).
calculer_heures_disponibles_par_jour(w)	Calcule le nombre d'heures de travail effectif par jour d'un worker (durée journée - 1h pause).
peut_effectuer_field_dans_periode(f, w, h)	Vérifie si un champ f peut être récolté par w dans h heures disponibles.
generer_emploi_du_temps()	Génère le planning complet sous forme de liste de listes [worker_id, field_id, jour].
calculer_nombre_jours_total(edt)	Retourne le nombre de jours total du planning en lisant le jour maximum.
generer_graphique(edt)	Génère et sauvegarde deux graphiques : répartition des champs par worker et diagramme de Gantt.
main()	Point d'entrée du programme. Instancie les classes, lance le planning et affiche les résultats.

7. Conclusion

Ce projet nous a permis de mettre en oeuvre les concepts fondamentaux de la programmation orientée objet en Python dans un contexte applicatif concret. La séparation en trois classes (Fields,

Workers, Calculs) offre une architecture claire et maintenable, où chaque classe a une responsabilité bien définie.

L'algorithme glouton adopté, bien que non optimal au sens mathématique, produit des résultats satisfaisants en 14 jours pour l'ensemble des 60 champs. Le tri préalable des champs par surface décroissante améliore l'utilisation des créneaux disponibles. La gestion des créneaux matin et après-midi garantit le respect de la contrainte de pause déjeuner et d'indivisibilité des champs.

Des améliorations pourraient être envisagées, notamment l'exploration d'algorithmes d'optimisation plus avancés (backtracking, programmation dynamique) pour réduire encore le nombre de jours, ou l'ajout d'une interface graphique pour visualiser le planning de façon interactive.
